Unit No. 3
CPU Scheduling

Course Outcome 1

- Apply scheduling algorithms to calculate turnaround time and average waiting time.

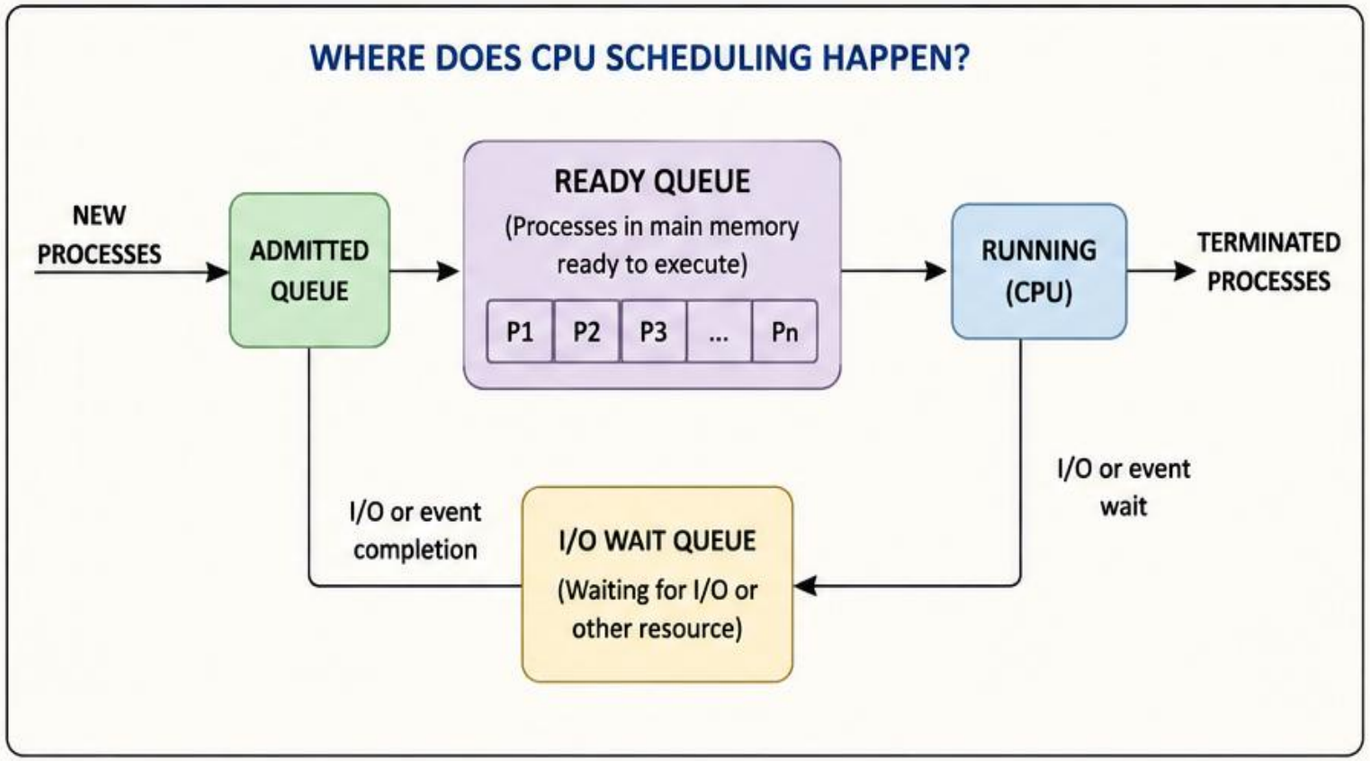
Content:

- **CPU Scheduling:** Concept, Criteria.
- **Scheduling Algorithms:** First-come, first-served (FCFS), Pre-emptive SJF, Non-Pre-emptive SJF, Round Robin (RR), Priority Scheduling.
- **Deadlocks:** Concept, conditions leading to deadlock, strategies to deal with deadlock
- **Deadlock Prevention**, Deadlock Avoidance, Detection and Recovery.

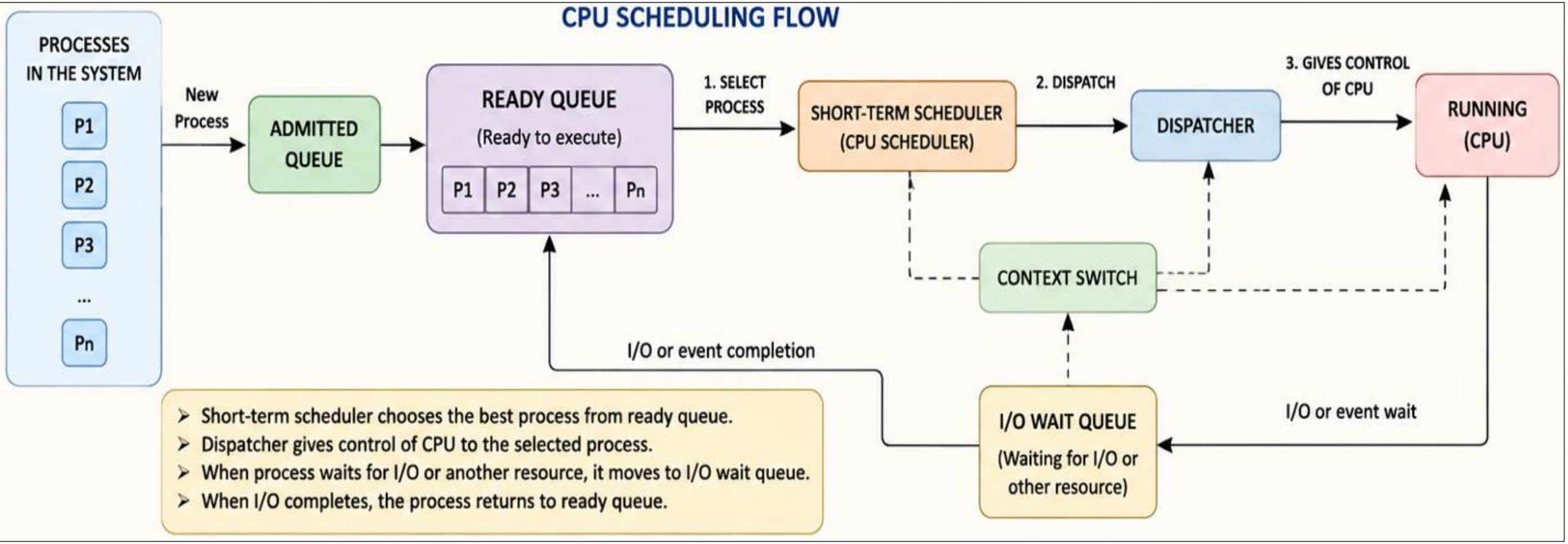
CPU Scheduling: Concept

CPU SCHEDULING – OVERVIEW

- **CPU scheduling** is a process which allows one process to use the CPU while the execution of another process is on hold (in waiting state) due to unavailability of any resource like I/O etc, thereby making full use of CPU.
- The aim of **CPU scheduling** is to make the system **efficient, fast and fair**.
- Whenever the **CPU becomes idle**, the operating system must select one of the processes in the **ready queue** to be executed.
- The selection process is carried out by the **short-term scheduler (or CPU scheduler)**.
- The scheduler selects from among the **processes in memory** that are ready to execute, and **allocates** the CPU to one of them.
- Another component involved in CPU scheduling function is the **dispatcher**.
- Dispatcher module **gives control of the CPU** to the process selected by the CPU scheduler.



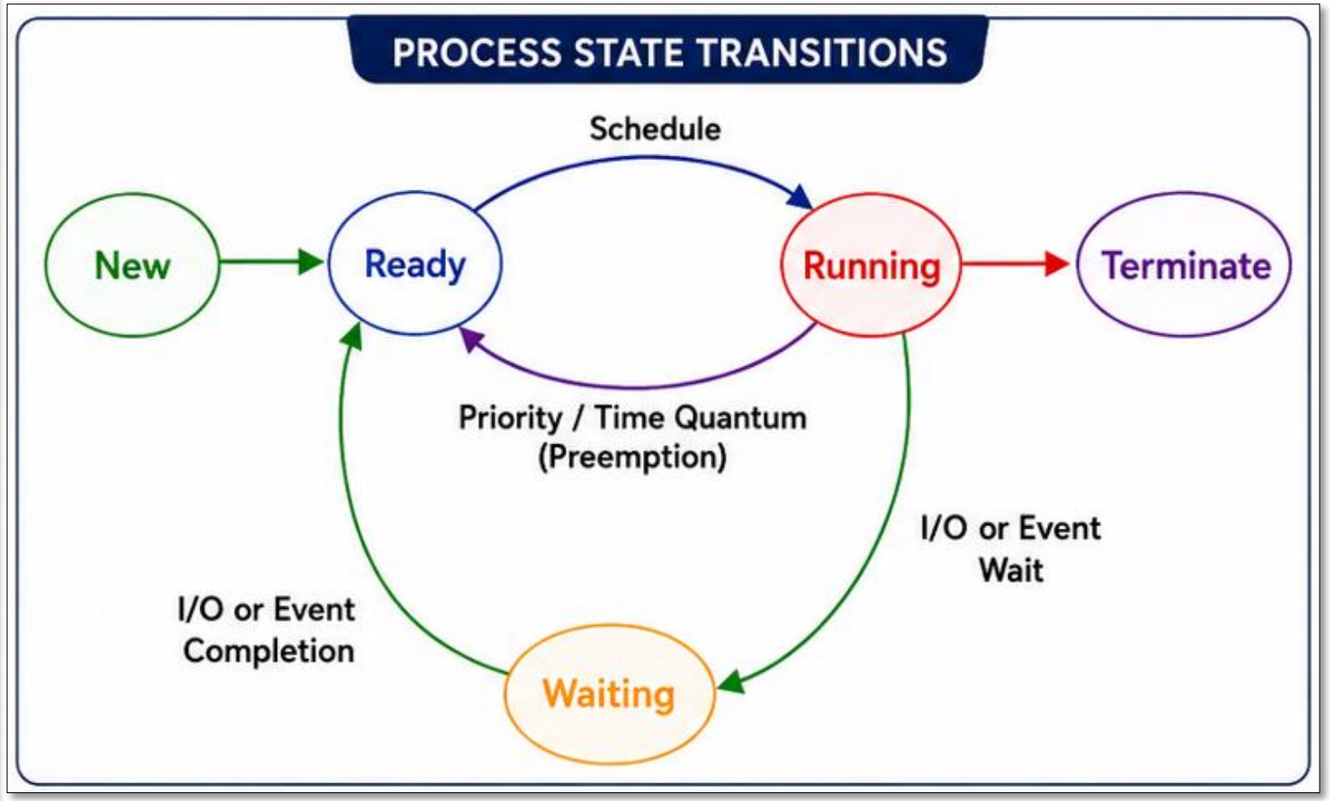
CPU Scheduling: Concept



CPU Scheduler

There are essential 4 conditions under which CPU scheduling decisions are taken:

1. If a process is making the switch between the running state to the waiting state (could be for an I/O request, or invocation of wait() for terminating one of its child processes)
2. If a process is making the switch from the running state to the ready state (on the occurrence of an interrupt, for example)
3. If a process is making the switch between waiting and ready state (e.g. when its I/O request completes)
4. If a process terminates upon completion of execution.



CPU Scheduler

- So in the case of **conditions 1 and 4**, the CPU does not really have a choice of scheduling, if a process exists in the ready queue the CPU's response to this would be to select it for execution.
- In **cases 2 and 3**, the CPU has a choice of selecting a particular process for executing next.

SUMMARY



1. Running → Waiting

Proces requests I/O or invokes wait() for a child process.



2. Running → Ready

Process is interrupted and moved back to ready state.



3. Waiting → Ready

I/O or event completes and process becomes ready again.



4. Terminate

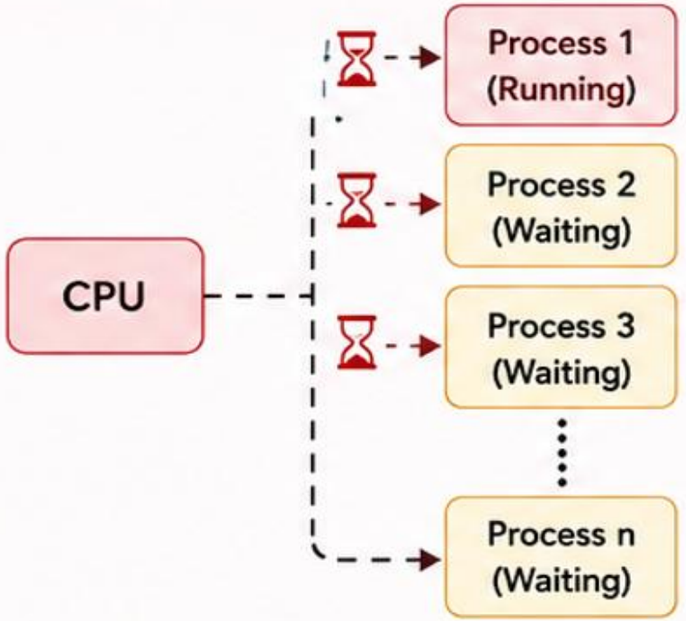
Process finishes execution and is removed from the system.

WHY WE PERFORM CPU SCHEDULING?

CPU Scheduling is the process of selecting a process from the **ready queue** and allocating the **CPU** to it.

WITHOUT CPU SCHEDULING

If one process uses the CPU for a long time or waits for I/O, other processes must wait for their turn.

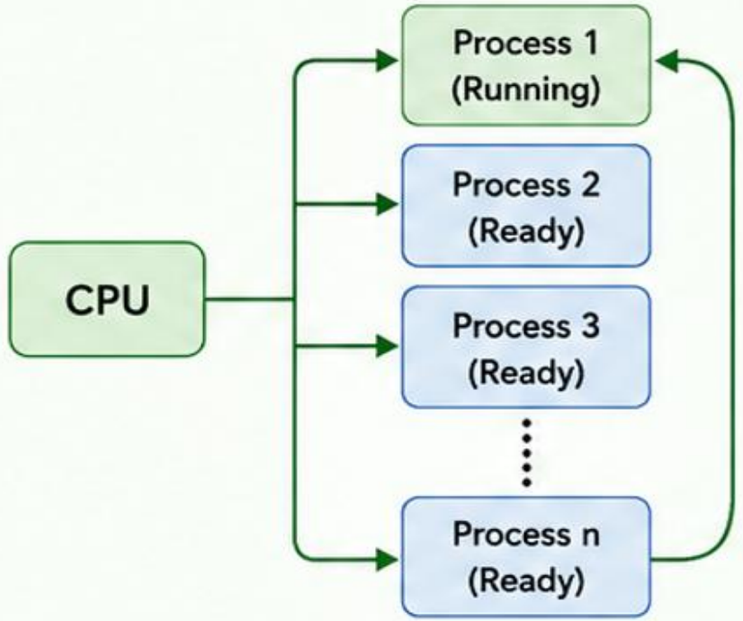


Results in poor CPU utilization, long waiting time and poor response.

VS

WITH CPU SCHEDULING

CPU scheduler selects the best process from the ready queue and gives CPU to it.



Results in high CPU utilization, less waiting time and good response.

OVERALL BENEFITS OF CPU SCHEDULING

MAIN REASONS



1. Maximize CPU Utilization

Keeps the CPU busy most of the time by selecting another process when the current one is waiting for I/O or another event.



2. Minimize Response Time

Ensures that processes get CPU time quickly, improving the system's responsiveness, especially in interactive systems.



3. Minimize Waiting Time

Reduces the time a process spends in the ready queue before it gets the CPU.



4. Ensure Fairness

Allows all processes a fair chance to use the CPU according to the scheduling policy.



5. Improve Throughput

Completes more processes in a given time by keeping the CPU productively busy.

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS
MIT-WPU

There are mainly two types of CPU scheduling

CPU Scheduling

Preemptive Scheduling
(CPU can be taken away from a process before it completes)

Priority Scheduling
Process with highest priority is executed first.

Longest Remaining Job First
Process with the longest remaining time is executed first.

Shortest Remaining Job First
Process with the shortest remaining time is executed first.

Round Robin
Each process gets a fixed time (time quantum) in a cyclic order.

Non-Preemptive Scheduling
(Once CPU is allocated to a process, it cannot be taken away until it completes)

First Come First Serve (FCFS)
Process that arrives first is executed first.

Longest Job First
Process with the longest CPU burst time is executed first.

Shortest Job First
Process with the shortest CPU burst time is executed first.

Highest Response Ratio Next
Process with the highest response ratio is executed next.

Preemptive vs. Non-Preemptive

Feature	Preemptive Scheduling	Non-Preemptive Scheduling
Meaning	CPU can be taken away from a running process	CPU is not taken away until the process finishes or enters the waiting state
CPU control	OS can interrupt the running process	Running process keeps the CPU until it voluntarily releases it
Process transition	Running → Ready **OR** Waiting → Ready	Running → Waiting **OR** Running → Terminated
Response time	Generally faster	Generally slower
Context switching	More frequent	Less frequent

Preemptive vs. Non-Preemptive

Feature	Preemptive Scheduling	Non-Preemptive Scheduling
Overhead	Higher due to more context switches	Lower
Suitable for	Interactive and time-sharing systems	Batch-oriented systems
Examples	Round Robin, SRTF, Preemptive Priority	FCFS, SJF, Non-Preemptive Priority

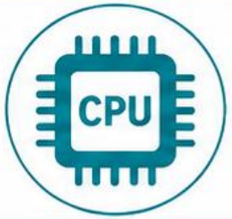
⭐ Easy way to remember

Preemptive = "CPU can be taken away."

Non-Preemptive = "CPU cannot be taken away until the process releases it."

Scheduling Criteria

CPU PERFORMANCE METRICS



1. CPU UTILIZATION

Keep the CPU as busy as possible.

- CPU utilization aims to maximize the CPU's busy time by minimizing idle periods, ensuring the processor is working as much as possible.



2. THROUGHPUT

Measure of work done.

- Throughput is the number of processes that complete their execution per unit of time. When the CPU is busy executing processes, useful work is accomplished.



3. TURNAROUND TIME

Time from submission to completion.

- Turnaround time is the total time taken to execute a particular process. It is calculated as the interval between the time of submission of the process and the time of its completion.



4. WAITING TIME

Time spent waiting in the ready queue.

- Waiting time is the total time a process spends in the ready queue waiting for the CPU to be allocated.



5. RESPONSE TIME

Time until first response is produced.

- Response time is the time elapsed from when a request (or process) is submitted until the system produces the first response.



These metrics help in evaluating and improving CPU scheduling and overall system performance.



Scheduling Criteria

SCHEDULING ALGORITHM OPTIMIZATION CRITERIA

For any algorithm, to be best, following Scheduling Algorithm Optimization Criteria are followed:



Max CPU utilization

Keep the CPU busy as much as possible.



Max throughput

Maximize the number of processes completed per unit time.



Min turnaround time

Minimize the total time from submission of a process to its completion.



Min waiting time

Minimize the total time processes spend waiting in the ready queue.



Min response time

Minimize the time from submission to the first response.



Overall Goal

Improve system performance and ensure efficient use of CPU and resources.



The goal of scheduling is to optimize these criteria to achieve high performance and good user experience.

CPU SCHEDULING ALGORITHMS

1.	**FCFS** (First Come First Served Scheduling) - Non Preemptive	Processes are executed in the order of their arrival in the ready queue. The process that comes first is served first.
2.	**SJF** (Shortest Job First) - Non Preemptive	The process with the shortest burst time is selected next for execution. Once a process starts, it runs until completion.
3.	**SRTN** (Shortest Remaining Time) - Preemptive SJF	The process with the shortest remaining time is executed. If a new process arrives with less remaining time, the current process is preempted.
4.	**RR** (Round Robin)	Each process gets a fixed time quantum (time slice). If the process does not finish in the given time, it is moved to the end of the queue.
5.	**Priority Scheduling**	Processes are executed based on priority. The process with the highest priority is executed first. (Preemptive or Non-Preemptive based on implementation)
6.	**Multilevel Queue Scheduling**	Ready queue is divided into multiple queues based on process type. Each queue has its own scheduling algorithm. High priority queues are served first.

These algorithms help the operating system to allocate CPU to processes in an efficient and fair manner.

Scheduling Criteria

SCHEDULING ALGORITHM BASICS

Below are different time values with respect to a process, which need to be calculated during every algorithm.

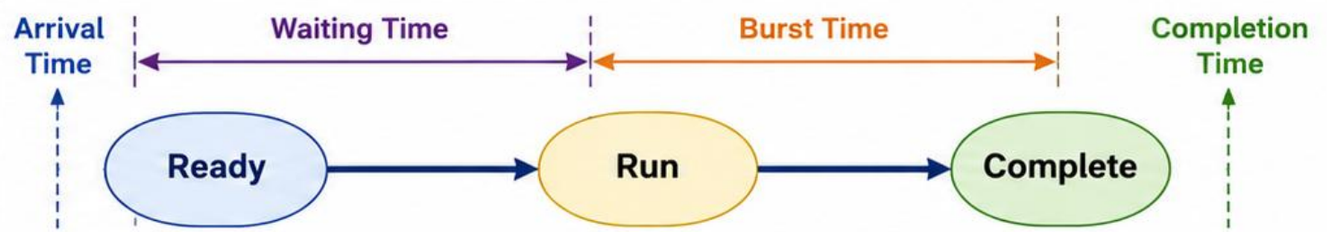
Arrival Time: Time at which the process arrives in the ready queue.

Completion Time: Time at which the process completes its execution.

Burst Time: Time required by a process for CPU execution.

Turn Around Time: Time Difference between completion time and arrival time.

Turn Around Time = Completion Time – Arrival Time



- **Arrival Time**: When the process enters the ready queue.
- **Waiting Time**: Time spent in the ready queue waiting for CPU.
- **Burst Time**: Actual CPU execution time of the process.
- **Completion Time**: When the process finishes execution.

These time values are essential to evaluate and compare scheduling algorithms for better performance.

Scheduling Algorithm Basics

For all algorithms: To solve a sum:

Step 1: Plot Gantt chart

Step 2: Find out Completion time of all process from chart

Step 3: Calculate Turn around time as Completion Time – Arrival Time (which is given in sum)

Step 4: Calculate Waiting Time as Turn around time – Burst time (again which is given in sum)

1.First- Come, First-Served (FCFS) Scheduling

FCFS is the **simplest CPU scheduling algorithm.** Processes are executed in the **order they arrive** in the ready queue.

FCFS algorithm allocates the CPU to the process requesting the CPU first, then to the second requesting process and so on.

Performs scheduling based on the **arrival times** of processes.

It uses the **First-In-First-Out (FIFO)** queue for this purpose.

New processes are attached at the **rear (tail)** of the queue whereas, processes are removed from the **head (front)** of the queue.

It is implemented as **non-preemptive** scheduling algorithm.

1.First- Come, First-Served (FCFS) Scheduling

Advantages of FCFS algorithm

- Simplest of all scheduling algorithms
- Easy to understand
- Easy to implement
- Performs better with processes having longer burst time
- Free from starvation: every process that wants to access the CPU, will be allocated the CPU after all the processes arriving before it have got their turn on the CPU. (No indefinite waiting for CPU)

1.First- Come, First-Served (FCFS) Scheduling

Disadvantages of FCFS algorithm

- Waiting time of processes increases as compared to other algorithms.
- Less efficient than other scheduling algorithms.
- **Convoy Effect**: A long process makes all smaller processes to wait for its completion.
- It performs better for the processes with longer burst time.
- Favours CPU bound processes over I/O bound processes.
- Not suitable for time-sharing system where each user expects an interactive system with regular time intervals.

Activate Windows
Go to Settings to activate Windo

1. First- Come, First-Served (FCFS) Scheduling- Example

First- Come, First-Served (FCFS) Scheduling

(Non-preemptive)

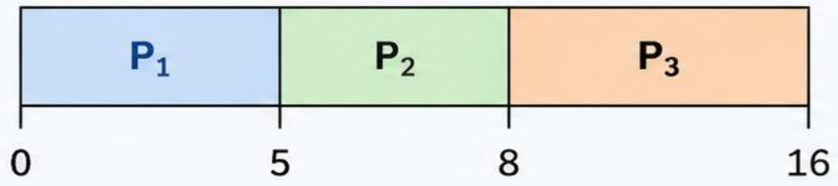
Q. Consider the following table of arrival time and burst time for three processes P1, P2 and P3

Process	Arrival Time	Burst Time
p1	0	5
p2	0	3
p3	0	8

1. First- Come, First-Served (FCFS) Scheduling- Example

FCFS (First-Come, First-Served) Scheduling – Solution

GANTT CHART (at t = 16)



FORMULAS USED

- Turnaround Time (TAT) = Completion Time – Arrival Time
- Waiting Time (WT) = Turnaround Time – Burst Time

AVERAGE RESULTS

- Average Turnaround Time = 9.67
- Average Waiting Time = 4.33

Processes	AT (Arrival Time)	BT (Burst Time)	CT (Completion Time)	TAT (Turnaround Time)	WT (Waiting Time)
P_1	0	5	5	5 - 0 = 5	5 - 5 = 0
P_2	0	3	8	8 - 0 = 8	8 - 3 = 5
P_3	0	8	16	16 - 0 = 16	16 - 8 = 8

FCFS is simple and fair as it executes processes in the order they arrive, but it may lead to higher waiting time for longer jobs.

1. First- Come, First-Served (FCFS) Scheduling- Example

Q. Consider the following table of arrival time and burst time for three processes P1, P2 and P3

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	2 ms
P2	3 ms	0 ms
P3	4 ms	4 ms

1. First- Come, First-Served (FCFS) Scheduling- Example

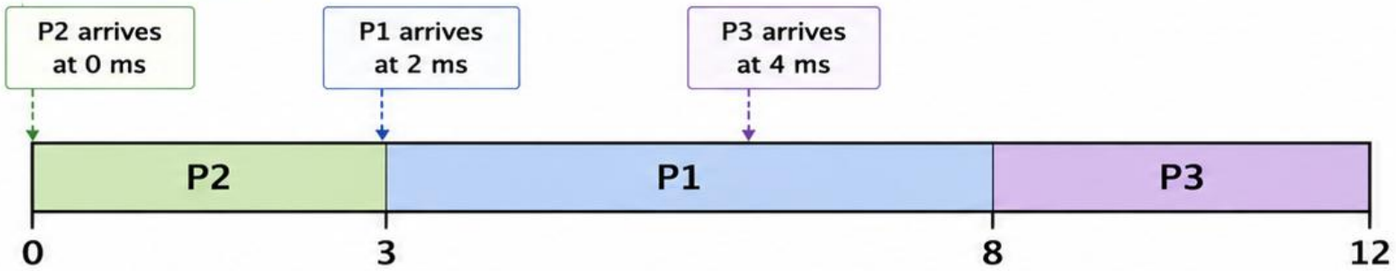
FCFS (First-Come, First-Served) Scheduling – Solution

In FCFS, processes are executed in the order of their arrival time.

1. Given Processes

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	2 ms
P2	3 ms	0 ms
P3	4 ms	4 ms

2. Gantt Chart



3. Calculation of Times

Process	AT (ms)	BT (ms)	CT (Completion Time) (ms)	TAT = CT – AT (ms)	WT = TAT – BT (ms)
P2	0	3	3	3 – 0 = 3	3 – 3 = 0
P1	2	5	8	8 – 2 = 6	6 – 5 = 1
P3	4	4	12	12 – 4 = 8	8 – 4 = 4

Where, CT = Completion Time, TAT = Turnaround Time, WT = Waiting Time

Average Turnaround Time (TAT)

$$= \frac{3 + 6 + 8}{3} = \frac{17}{3} = 5.67 \text{ ms}$$

Average Waiting Time (WT)

$$= \frac{0 + 1 + 4}{3} = \frac{5}{3} = 1.67 \text{ ms}$$

Therefore, Average Turnaround Time = 5.67 ms | Average Waiting Time = 1.67 ms

1. First- Come, First-Served (FCFS) Scheduling- Example

Q. Consider the following table of arrival time and burst time for three processes P1 to P5

Processes	Arrival Time	Burst Time
P1	0	4
P2	1	3
P3	2	1
P4	3	2
P5	4	5

1. First- Come, First-Served (FCFS) Scheduling- Example

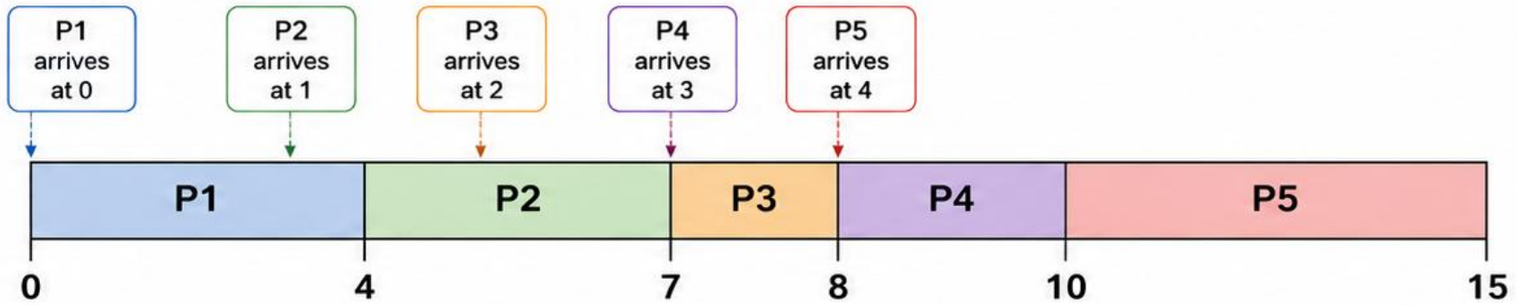
FCFS (First-Come, First-Served) Scheduling – Solution

In FCFS, processes are executed in the order of their arrival time.

1. Given Processes

Processes	Arrival Time (AT)	Burst Time (BT)
P1	0	4
P2	1	3
P3	2	2
P4	3	2
P5	4	5

2. Gantt Chart



3. Calculation of Times

Process	AT	BT	Completion Time (CT)	Turnaround Time (TAT) CT – AT	Waiting Time (WT) TAT – BT
P1	0	0	4	4 – 0 = 4	4 – 4 = 0
P2	1	3	7	7 – 1 = 6	6 – 3 = 3
P3	2	1	8	8 – 2 = 6	6 – 1 = 5
P4	3	2	10	10 – 3 = 7	7 – 2 = 5
P5	4	5	15	15 – 4 = 11	11 – 5 = 6

Average Turnaround Time (TAT)

$$= \frac{4 + 6 + 6 + 7 + 11}{5} = \frac{34}{5} = 6.80$$

Average Waiting Time (WT)

$$= \frac{0 + 3 + 5 + 5 + 6}{5} = \frac{19}{5} = 3.80$$

Therefore, Average Turnaround Time = 6.80 | Average Waiting Time = 3.80

1. First- Come, First-Served (FCFS) Scheduling- Example

First- Come, First-Served (FCFS) Scheduling

(Non-preemptive)

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
  The Gantt Chart for the schedule is:



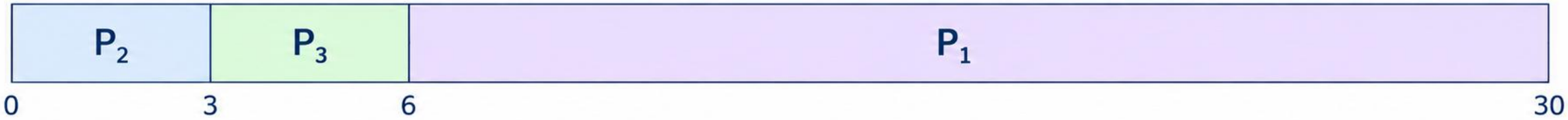
- Waiting time for P_1 = 0ms; P_2 = 24ms; P_3 = 27ms
- Average waiting time: (0 + 24 + 27)/3 = 17ms

1. First- Come, First-Served (FCFS) Scheduling- Example

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order: P_2 , P_3 , P_1

- The Gantt chart for the schedule is:



- Waiting time for P_1 = 6ms; P_2 = 0ms; P_3 = 3ms
- Average waiting time: (6 + 0 + 3) / 3 = 3ms
- Much better than previous case

➡ **Convoy effect** - short process behind long process
- Consider one CPU-bound and many I/O-bound processes

2. Shortest-Job-First (SJF) Scheduling

Basic Concept

- Each process is associated with the **length of its next CPU burst**.
- The CPU is assigned to the process with the **shortest CPU burst time**.
- SJF is an **optimal scheduling algorithm** because it minimizes the **average waiting time** for a given set of processes.
- SJF selects the process with the **smallest burst time** from the ready queue to execute next.

◆ **Types of SJF**

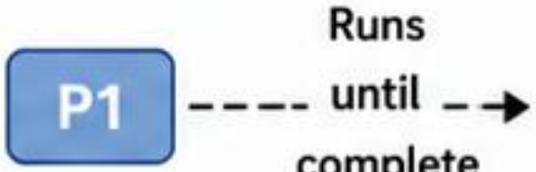
- **Non-Preemptive SJF:** Once a process starts execution, it runs until completion.
- **Preemptive SJF:** Called **Shortest Remaining Time First (SRTF)**; a newly arrived process can **preempt** the currently running process if it has a shorter remaining burst time.

2. Shortest-Job-First (SJF) Scheduling

2. TYPES OF SJF

(A) Non-Preemptive SJF

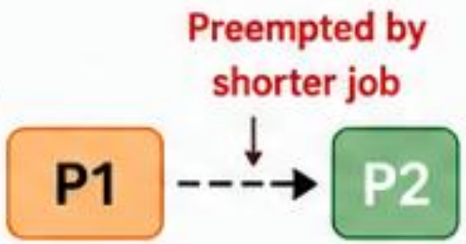
- Once a process starts execution, it runs until completion.



(B) Preemptive SJF

(Shortest Remaining Time First - SRTF)

- A newly arrived process can preempt the currently running process if it has a shorter remaining burst time.



2. Shortest-Job-First (SJF) Scheduling

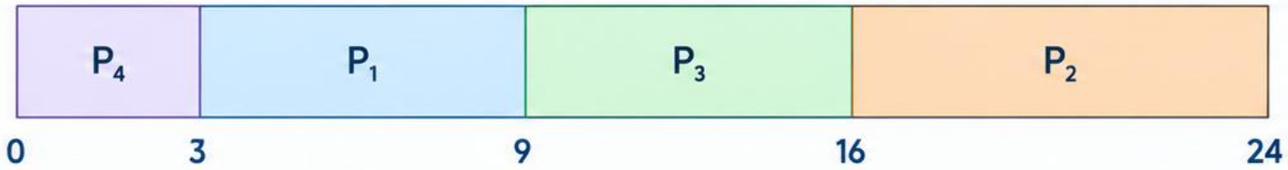
Example of SJF (Shortest Job First) Scheduling

1. Given Data

Process	Burst Time (ms)
P_1	6
P_2	8
P_3	7
P_4	3

- All processes arrive at time 0.
- SJF selects the process with the shortest burst time first.

2. Gantt Chart



3. Waiting Time and Turnaround Time

Process	Burst Time (ms)	Completion Time (ms)	Waiting Time (ms)	Turnaround Time (ms)
P_4	3	3	0	3
P_1	6	9	3	9
P_3	7	16	9	16
P_2	8	24	16	24

Results

Average Waiting Time
7 ms

Average Turnaround Time
13 ms

2. Shortest-Job-First (SJF) Scheduling

Example :-

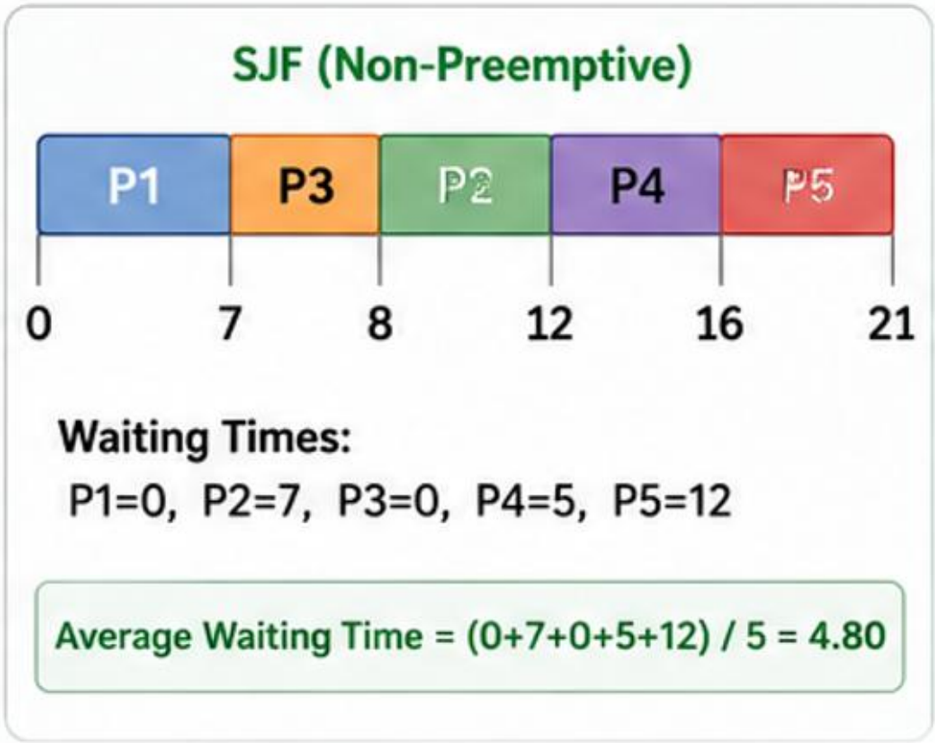
Gantt Chart :-

2. Shortest-Job-First (SJF) Scheduling

Example :-

Processes (AT = Arrival Time)		
Process	Arrival Time	Burst Time
P1	0	7
P2	1	4
P3	2	1
P4	3	4
P5	4	5

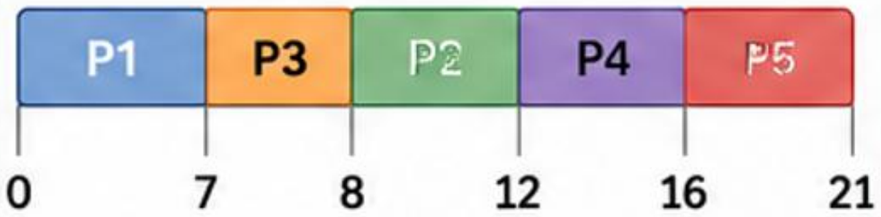
Gantt Chart :-



2. Shortest-Job-First (SJF) Scheduling

Gantt Chart :-

SJF (Non-Preemptive)

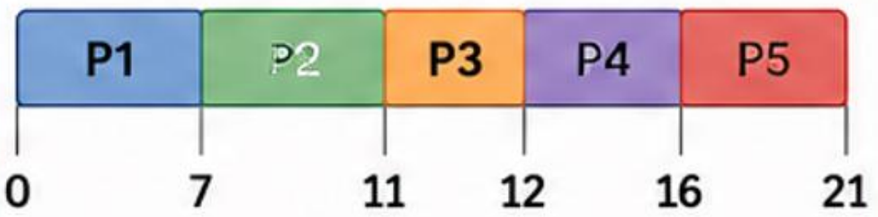


Waiting Times:

P1=0, P2=7, P3=0, P4=5, P5=12

Average Waiting Time = (0+7+0+5+12) / 5 = 4.80

FCFS (For Comparison)



Waiting Times:

P1=0, P2=6, P3=9, P4=9, P5=12

Average Waiting Time = (0+6+9+9+12) / 5 = 7.20

SJF gives lower average waiting time (4.80) compared to FCFS (7.20).

SJF is optimal – it gives the minimum average waiting time for a given set of processes.

2. Shortest-Job-First (SJF) Scheduling

Example :-

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Step-by-step:

- At t=0, only P1 has arrived → run P1 (0 to 8). (Even though P2/P4 are shorter, they haven't arrived yet, and non-preemptive means we can't interrupt P1 anyway once it starts.)
- At t=8, P2, P3, P4 have all arrived (burst: 4, 9, 5) → pick shortest = P2 (8 to 12)
- At t=12, remaining: P3(9), P4(5) → pick P4 (12 to 17)
- At t=17, remaining: P3(9) → run P3 (17 to 26)

2. Shortest-Job-First (SJF) Scheduling

Example :-

Gantt Chart :-

2. Shortest-Job-First (SJF) Scheduling

Solution:-

Gantt Chart:

```
| P1 | P2 | P4 | P3 |
0    8    12   17   26
```

Process	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	1	4	12	11	7
P4	3	5	17	14	9
P3	2	9	26	24	15

- **Average WT** = (0+7+9+15)/4 = **7.75**
- **Average TAT** = (8+11+14+24)/4 = **14.25**

2. Shortest-Job-First (SJF) Scheduling

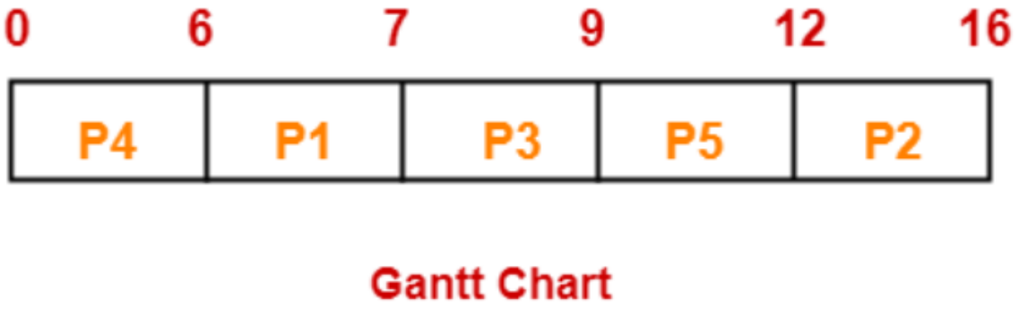
Example:- Consider the set of 5 processes whose arrival time and burst time are given below-

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

If the CPU scheduling policy is SJF non-preemptive, calculate the average waiting time and average turnaround time.

2. Shortest-Job-First (SJF) Scheduling

Solution:-



Gantt Chart

Process Id	Exit time	Turn Around time	Waiting time
P1	7	7 – 3 = 4	4 – 1 = 3
P2	16	16 – 1 = 15	15 – 4 = 11
P3	9	9 – 4 = 5	5 – 2 = 3
P4	6	6 – 0 = 6	6 – 6 = 0
P5	12	12 – 2 = 10	10 – 3 = 7

Now,

- Average Turn Around time = (4 + 15 + 5 + 6 + 10) / 5 = 40 / 5 = 8 unit
- Average waiting time = (3 + 11 + 3 + 0 + 7) / 5 = 24 / 5 = 4.8 unit

3. Shortest Remaining Time First (SRTF) Scheduling

2. Preemptive SJF (SRTF — Shortest Remaining Time First)

Here, if a new process arrives with a **shorter remaining burst time** than the currently running process, the CPU **switches** to it immediately.

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

3. Shortest Remaining Time First (SRTF) Scheduling

Step-by-step (tracking remaining time):

- t=0: Only P1 present → run P1. Remaining BT(P1) = 8
- t=1: P2 arrives (BT=4). Compare remaining P1 (7) vs P2 (4) → P2 is shorter → **switch to P2**
- t=2: P3 arrives (BT=9). Current process P2 has remaining 3 → still shorter → continue P2
- t=3: P4 arrives (BT=5). P2 remaining = 2 → still shortest → continue P2
- t=5: P2 finishes. Remaining: P1(7), P3(9), P4(5) → shortest is P4 → run P4
- t=10: P4 finishes. Remaining: P1(7), P3(9) → run P1
- t=17: P1 finishes. Remaining: P3(9) → run P3
- t=26: P3 finishes

3. Shortest Remaining Time First (SRTF) Scheduling

Solution:-

Gantt Chart :-

3. Shortest Remaining Time First (SRTF) Scheduling

Gantt Chart:

```
| P1 | P2 | P4 | P1 | P3 |
0    1    5    10   17   26
```

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

- **Average WT** = (9+0+15+2)/4 = **6.5**
- **Average TAT** = (17+4+24+7)/4 = **13.0**

3. Shortest Remaining Time First (SRTF) Scheduling

Example 2 :- Consider the set of 5 processes whose arrival time and burst time are given below-

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

If the CPU scheduling policy is SJF Preemptive, calculate the average waiting time and average turnaround time.

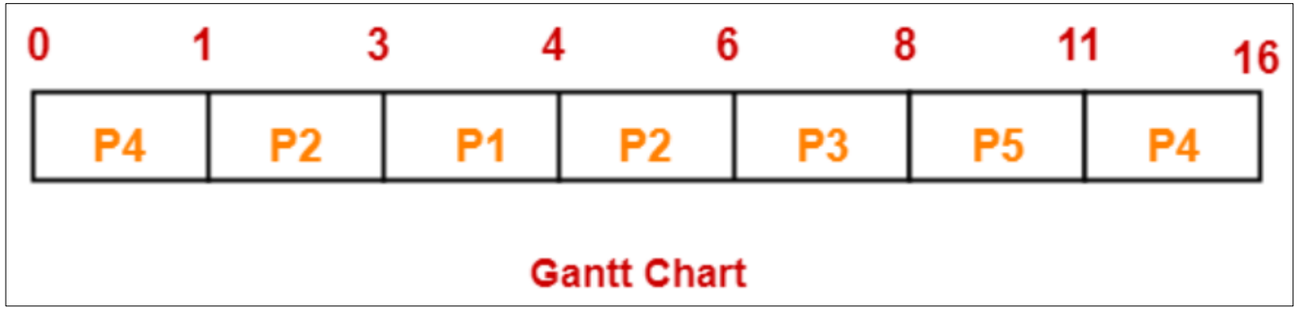
3. Shortest Remaining Time First (SRTF) Scheduling

Solution:-

Gantt Chart :-

3. Shortest Remaining Time First (SRTF) Scheduling

Solution:-



Gantt Chart

- Average Turn Around time = (1 + 5 + 4 + 16 + 9) / 5 = 35 / 5 = 7 unit
- Average waiting time = (0 + 1 + 2 + 10 + 6) / 5 = 19 / 5 = 3.8 unit

Process Id	Exit time	Turn Around time	Waiting time
P1	4	4 – 3 = 1	1 – 1 = 0
P2	6	6 – 1 = 5	5 – 4 = 1
P3	8	8 – 4 = 4	4 – 2 = 2
P4	16	16 – 0 = 16	16 – 6 = 10
P5	11	11 – 2 = 9	9 – 3 = 6

4. Round Robin Scheduling

- **Why was Round Robin introduced?**

  The key problem was:
  - **How can multiple users/processes share one CPU without allowing one process to monopolize it?**
  - Round Robin solved this by giving every process a **small, equal CPU time slice**.

- Round Robin Scheduling is a method used by operating systems to manage the execution time of multiple processes that are competing for CPU attention.
- It is called "round robin" because the system rotates through all the processes, allocating each of them a fixed time slice or "quantum", regardless of their priority.



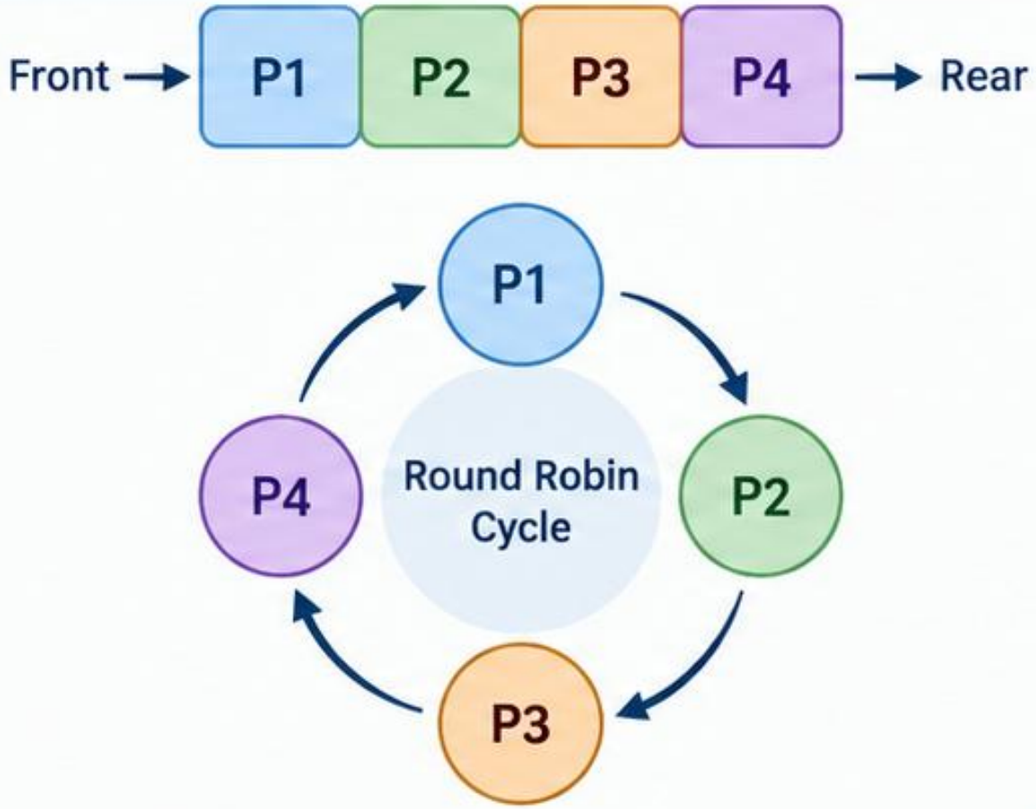
A time-sharing CPU scheduling algorithm that allocates a small unit of CPU time to each process in a cyclic order.

4. Round Robin Scheduling

How it Works

- Each process gets a small unit of CPU time (**time quantum** q), usually 10–100 milliseconds.
- After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q, then each process gets $1/n$ of the CPU time in chunks of at most q time units. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule the next process.

Ready Queue (Example)



4. Round Robin Scheduling

Example:

To understand the Round Robin Scheduling algorithm, consider the following table of arrival times and burst times for three processes, P1, P2, and P3, and given **Time Quantum = 4 ms, calculate the average TAT & average WT**

Process	Burst Time
P_1	24
P_2	3
P_3	3

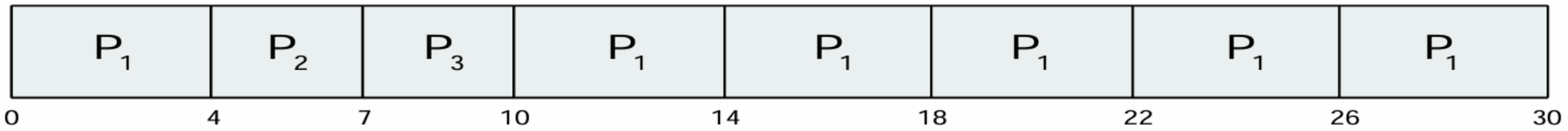
4. Round Robin Scheduling

Example:

Process	Burst Time
P_1	24
P_2	3
P_3	3

4. Round Robin Scheduling

Solution: The Gantt chart is:



Process	Burst Time	Completion time	Turnaround Time = C.T.- A.T.	Waiting Time = TAT - BT
P1	24	30	30	6
P2	3	7	7	4
P3	3	10	10	7
		Avg.	**= 47/3 = 15.67**	**= 17/3 = 5.67**

4. Round Robin Scheduling

Example:

To understand the Round Robin Scheduling algorithm, consider the following table of arrival times and burst times for three processes, P1, P2, and P3, and given **Time Quantum = 2 ms, calculate the average TAT & average WT**

Process	Burst Time	Arrival Time
P1	4 ms	0 ms
P2	5 ms	0 ms
P3	3 ms	0 ms

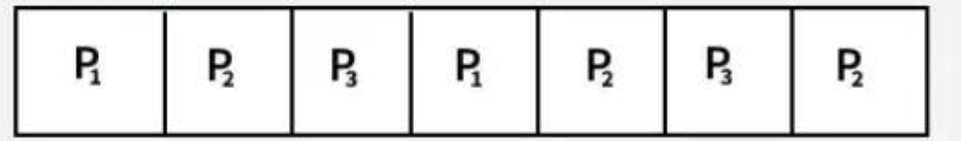
4. Round Robin Scheduling

Solution:-

4. Round Robin Scheduling

Solution:-

Gantt chart



Processes	AT	BT	CT	TAT	WT
P1	0	4	8	8-0 = 8	8-4 = 4
P2	0	5	12	12-0 = 12	12-5 = 7
P3	0	3	11	11-0 = 11	11-3 = 8

- **Average Turn around time** = (8 + 12 + 11)/3 = 31/3 = 10.33 ms
- **Average waiting time** = (4 + 7 + 8)/3 = 19/3 = 6.33 ms

4. Round Robin Scheduling

Example:

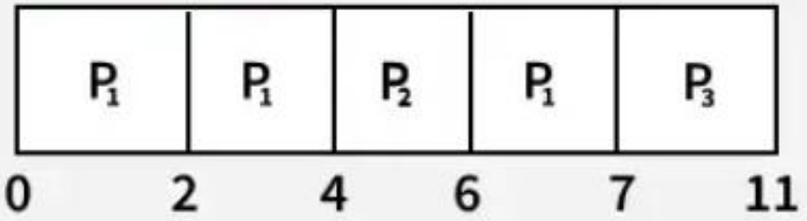
To understand the Round Robin Scheduling algorithm, consider the following table of arrival times and burst times for three processes, P1, P2, and P3, and given **Time Quantum = 2 ms, calculate the average TAT & average WT**

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	0 ms
P2	2 ms	4 ms
P3	4 ms	5 ms

4. Round Robin Scheduling

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	0 ms
P2	2 ms	4 ms
P3	4 ms	5 ms

Gantt chart



Process	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P1	7 ms	7 ms	2 ms
P2	6 ms	2 ms	0 ms
P3	11 ms	6 ms	2 ms

- **Average Turn around time** =7+2+6/3=15/3=5ms
- **Average waiting time** = 2+0+2/3=1.33ms

4. Round Robin Scheduling

Problem-01:

Consider the set of 5 processes whose arrival time and burst time are given below-

Process Id	Arrival time	Burst time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

If the CPU scheduling policy is Round Robin with time quantum = 2 unit, calculate the average waiting time and average turn around time.

4. Round Robin Scheduling

Solution:-

5. Priority Scheduling

- Priority Scheduling is a CPU scheduling algorithm used by an operating system to select the next process for execution based on its assigned priority.
- In this algorithm, each process is assigned a priority value. The CPU selects the process with the highest priority from the ready queue.
- Priority Scheduling can be:
  1. Non-Preemptive Priority Scheduling
  2. Preemptive Priority Scheduling

Important Note

Priority values can be assigned in two ways:

- **Smaller number = Higher priority:** Priority 1 is higher than priority 3.
- **Larger number = Higher priority:** Priority 5 is higher than priority 2.

5. Priority Scheduling

Example of Priority Scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

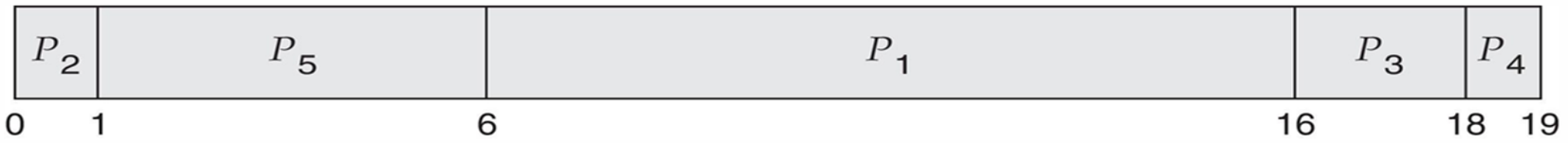
5. Priority Scheduling

Example of Priority Scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Solution:-

- Priority scheduling Gantt Chart



- Average waiting time = 8.2

5. Priority Scheduling

Example:-

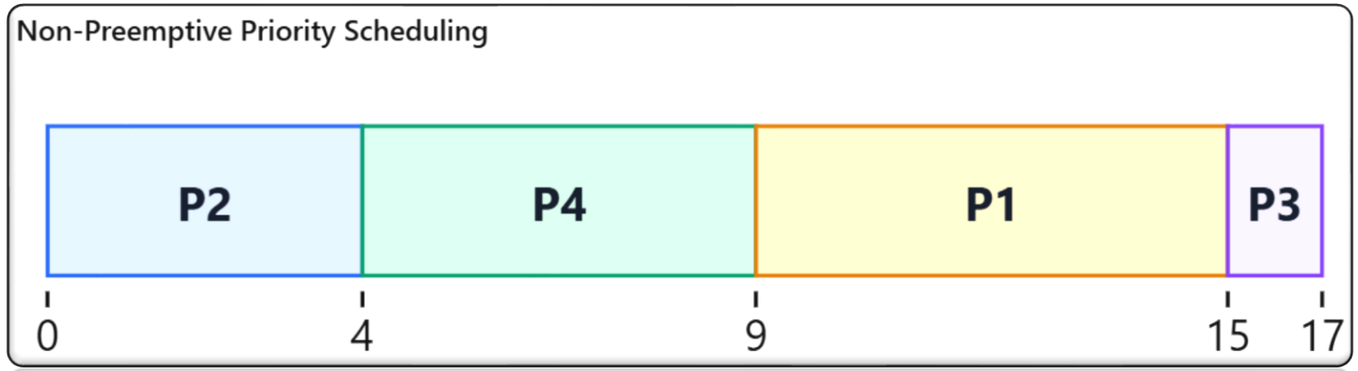
Process	Arrival Time	Burst Time	Priority			
P1	0	6	3			
P2	0	4	1			
P3	0	2	4			
P4	0	5	2			

Solution:-

5. Priority Scheduling

Example:-

Process	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	0	4	1
P3	0	2	4
P4	0	5	2



Solution:-

Process	BT	Priority	CT	WT	TAT
P2	4	1	4	0	4
P4	5	2	9	4	9
P1	6	3	15	9	15
P3	2	4	17	15	17

5. Priority Scheduling

Example:-

Non-Preemptive Priority Scheduling: Consider the following table of arrival time and burst time for three processes P1, P2 and P3:

Note: Lower number represents higher priority.

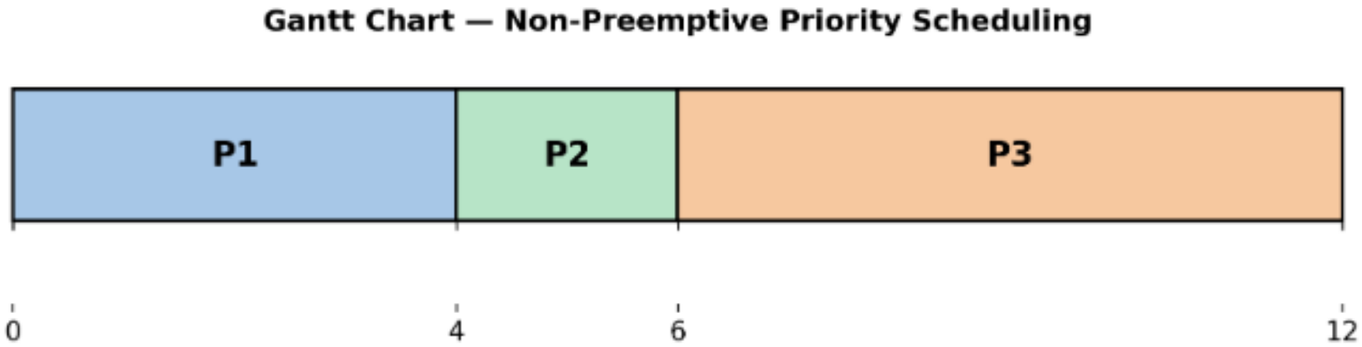
Process	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	2	1
P3	2	6	3

5. Priority Scheduling

Example:-

Process	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	2	1
P3	2	6	3

Solution:-



Process	Arrival	Burst	Priority	Completion	Turnaround	Waiting
P1	0	4	2	4	4 – 0 = 4	4 – 4 = 0
P2	1	2	1	6	6 – 1 = 5	5 – 2 = 3
P3	2	6	3	12	12 – 2 = 10	10 – 6 = 4

Average turnaround time = (4 + 5 + 10) / 3 = 19 / 3 ≈ **6.33**

Average waiting time = (0 + 3 + 4) / 3 = 7 / 3 ≈ **2.33**

5. Priority Scheduling

In **Preemptive Priority Scheduling**, the CPU can be taken away from the currently running process if a new process with a higher priority arrives.

Ex: A low-priority process is running, and a high-priority process arrives; the CPU immediately switches to the high-priority process.

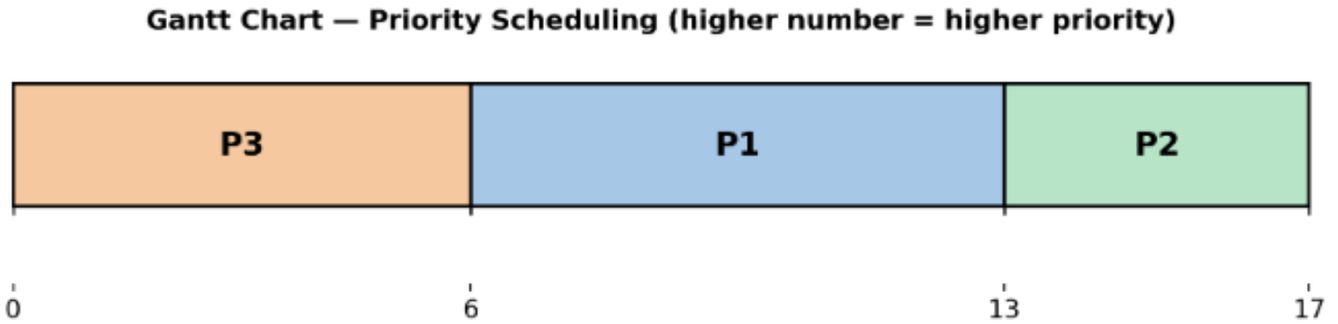
Example:- Example of Preemptive Priority Scheduling (Same Arrival Time)

Process	Arrival Time	Burst Time	Priority
P1	0	7	2
P2	0	4	1
P3	0	6	3

5. Priority Scheduling

Example:- **Example of Preemptive Priority Scheduling (Same Arrival Time)**

Process	Arrival Time	Burst Time	Priority
P1	0	7	2
P2	0	4	1
P3	0	6	3



Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT - AT)	Waiting Time (TAT - BT)
P1	0	7	13	13	6
P2	0	4	17	17	13
P3	0	6	6	6	0

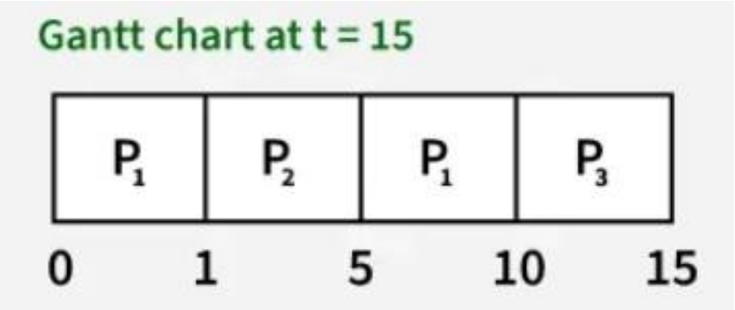
Average Turnaround Time = 12

Average Waiting Time = 6.33

5. Priority Scheduling

Example:- **Preemptive Priority Scheduling (Different Arrival Time)**

Process	Arrival Time	Burst Time	Priority
P1	0	6	2
P2	1	4	3
P3	2	5	1



Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT - AT)	Waiting Time (TAT - BT)
P1	0	6	10	10	4
P2	1	4	5	4	0
P3	2	5	15	13	8

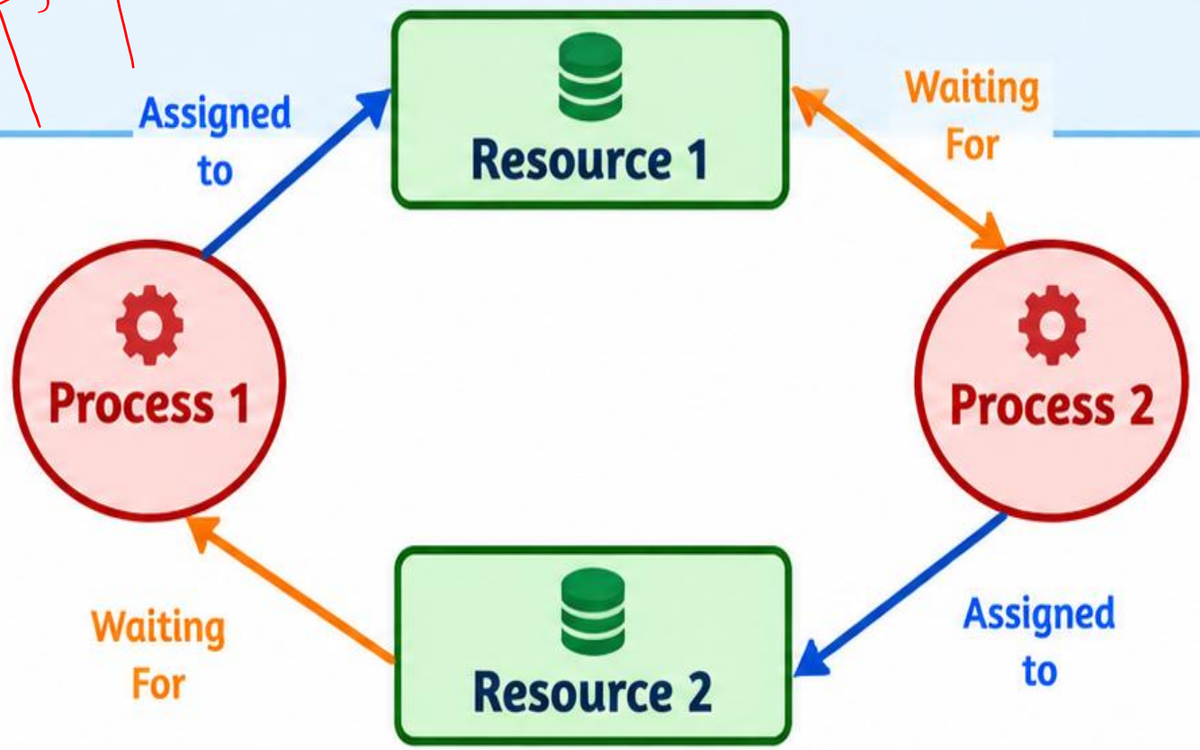
- **Average Turnaround Time = 9 Average Waiting Time = 4**

What is Deadlocks?

A **process** in **operating system** uses **resources** in the following way.

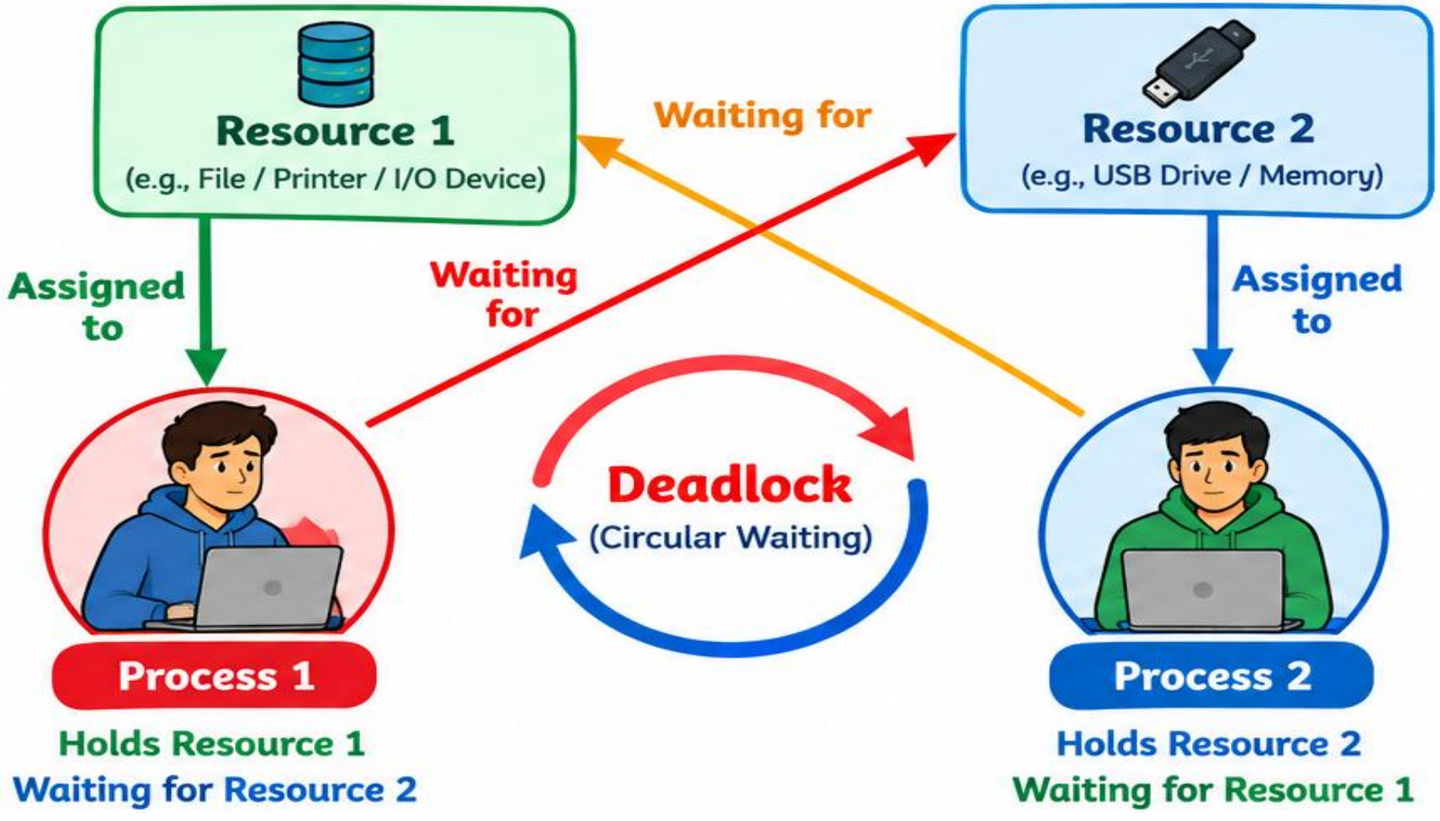
1. Requests a resource
2. Uses the resource
3. Releases the resource

A **deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.



What is Deadlock?

A **deadlock** occurs when two or more processes are permanently blocked because each process is waiting for a resource that is held by another process.



1 Process 1 has Resource 1
Process 1 holds Resource 1 (e.g., a file or printer).
It is waiting for Resource 2.

2 Process 2 has Resource 2
Process 2 holds Resource 2 (e.g., a USB drive).
It is waiting for Resource 1.

3 Both are blocked
Process 1 cannot continue until it gets Resource 2.
Process 2 cannot continue until it gets Resource 1.

4 Result: Deadlock
Both processes keep waiting for each other.
This creates a circular waiting condition, and neither process can proceed.
This situation is called **deadlock**.

Key Takeaway:
Deadlock = Holding one resource + Waiting for another resource + Circular waiting.

What is Deadlock?

- A deadlock is a situation in an operating system where a set of processes becomes permanently blocked because each process is waiting for a resource held by another process in the same set.
- In simple words, deadlock occurs when processes are waiting for one another, and none of them can proceed.

Example of deadlock

Consider two processes, P1 and P2, and two resources, R1 and R2.

- Process P1 holds resource R1 and waits for resource R2.
- Process P2 holds resource R2 and waits for resource R1.

Neither process can continue because each is waiting for a resource held by the other.

Necessary conditions for deadlock

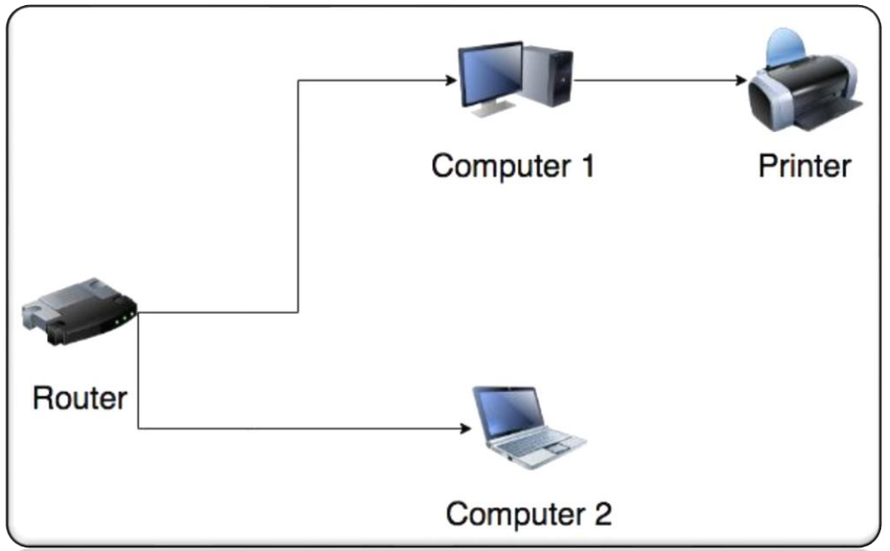
- According to the Coffman conditions, deadlock can occur only when all four of the following conditions hold simultaneously:
  1. Mutual Exclusion
  2. Hold and Wait
  3. No Preemption
  4. Circular Wait

Coffman conditions are the four necessary conditions identified by Edward G. Coffman and his colleagues in 1971 that must hold simultaneously for a deadlock to occur in a system of processes.

Necessary conditions for deadlock

1. Mutual Exclusion :

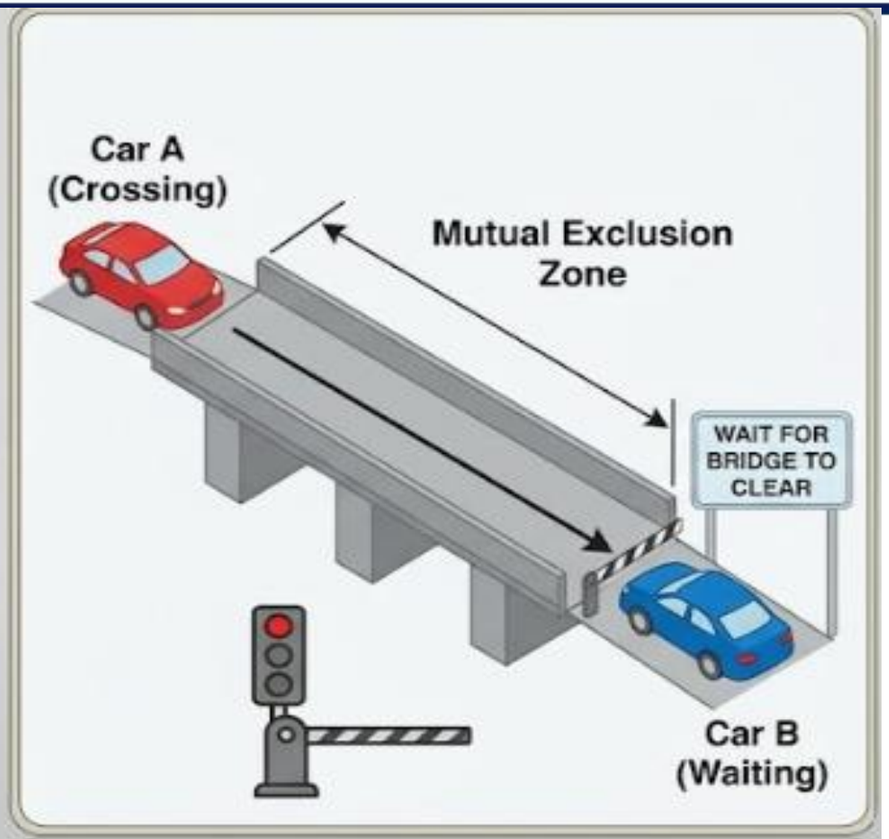
- Mutual exclusion means that at least one resource is non-shareable. Only one process can use that resource at a time.
- If a process is using a resource, another process must wait until the resource becomes available.
- **For example,** a printer printing a document cannot generally print two documents at the exact same time.
- If a resource is exclusively allocated to one process, it satisfies the mutual exclusion condition.



Necessary conditions for deadlock

1. Mutual Exclusion (Real-Life Example) :

- Consider a narrow, one-lane bridge in a rural area.
- Only one car can safely cross the bridge at a time from either direction.
- If a car (Process A) is crossing the bridge (the Resource), any other car arriving (Process B) must wait at the entrance until the bridge becomes free.
- The bridge can be considered a non-shareable resource; its exclusive use by one car at a time satisfies the mutual exclusion condition.
- Attempting to force two cars onto the bridge simultaneously can lead to blockages, just as attempting simultaneous resource access can lead to deadlocks.



2 Hold and Wait Condition

A process holds at least one resource while waiting to acquire additional resources held by other processes.

Definition

Hold and wait means that a process is holding at least one resource while waiting to acquire additional resources held by other processes. The process does not release the resources it already has.

Real-Life Example: Bank ATM

Consider a bank with two resources:

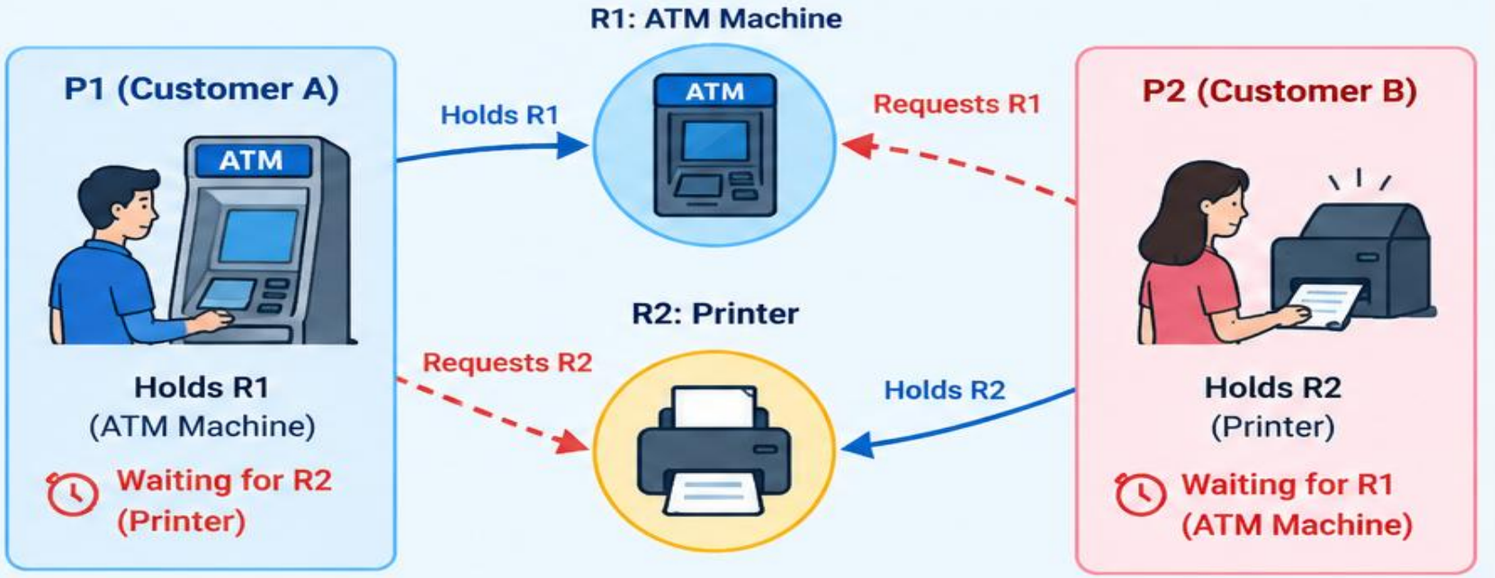
- R1 = ATM Machine (non-shareable)
- R2 = Cash **Deposit Slip** Printer (non-shareable)

Two customers (processes) **P1** and **P2** are using the bank's services.

Process Resource Table

Process	Currently holds	Waiting for
P1	R1 (ATM Machine)	R2 (Printer)
P2	R2 (Printer)	R1 (ATM Machine)

Scenario Illustration



P1 holds the ATM machine (R1) and waits for the printer (R2), while P2 holds the printer (R2) and waits for the ATM machine (R1). This is a hold and wait condition.

Key Takeaway

In this real-life example, both processes hold one resource and wait for another resource, **satisfying the hold and wait condition.**

4 No Preemption

A resource cannot be forcibly taken away from the process holding it.

Definition

No preemption means that a resource cannot be forcibly taken away from the process holding it. The process must release the resource voluntarily after completing its task or when the operating system permits release.

Explanation

- The operating system cannot simply take a resource from a process and give it to another process.
- The resource must be released by the process that currently owns it.

Key Point

No preemption does not mean a process never releases the resource, it means the resource cannot be forcibly taken away.

Example: Printer

Suppose process P1 is printing a document and process P2 also requests the printer.

1 P1 acquires the printer



Process P1 has acquired the printer and is printing a document.

2 P2 requests the printer



Process P2 also requests the printer.

3 Printer is occupied by P1



The printer is currently occupied by P1.

4 OS cannot take it away



The operating system cannot forcibly take the printer away from P1 in the middle of its operation.

5 P1 finishes and releases



P1 must finish printing and release the printer.

Conclusion

Therefore, the printer satisfies the no preemption condition in this example.

MIT-WPU

4 Circular Wait

A set of processes exists in which each process is waiting for a resource held by the next process in the cycle.

Definition

Circular wait means that a set of processes exists in which each process is waiting for a resource held by the next process in the cycle.

Explanation

Circular wait occurs when processes form a closed chain of waiting. Each process is waiting for a resource held by another process in that same chain.

Suitable Example: Three Processes

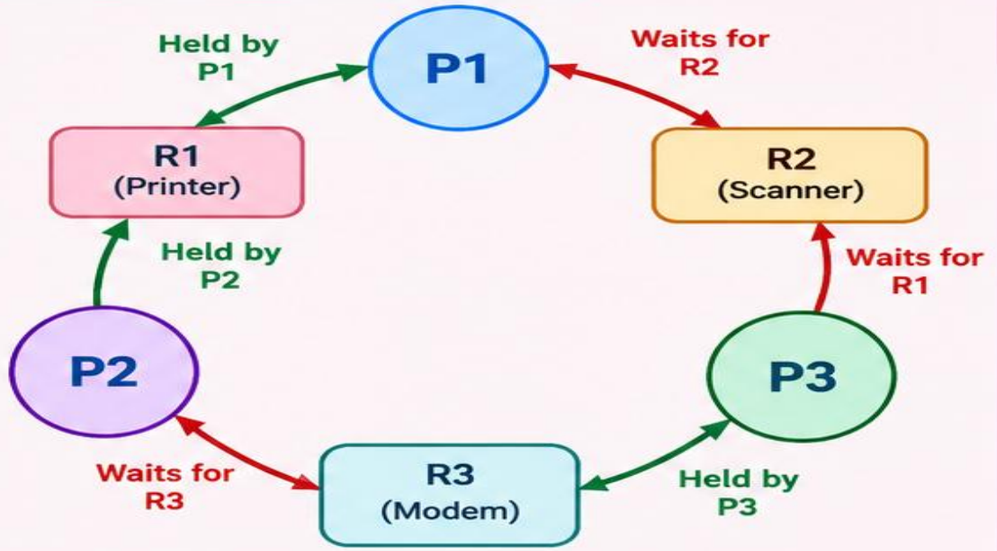
Consider three processes and three resources:

Process	Currently holds	Waiting for
P1	R1	R2
P2	R2	R3
P3	R3	R1

The waiting relationship:

- P1 waits for R2, which is held by P2.
- P2 waits for R3, which is held by P3.
- P3 waits for R1, which is held by P1.

Circular Wait Diagram



A set of processes waiting for each other in circular form.

Conclusion

Each process is waiting for a resource held by the next process in the cycle.
This closed chain of waiting satisfies the circular wait condition, which can lead to deadlock when all four Coffman conditions are present.

The 4 Necessary Conditions of Deadlock

All four conditions must be present at the same time for a deadlock to occur.

1 Mutual Exclusion

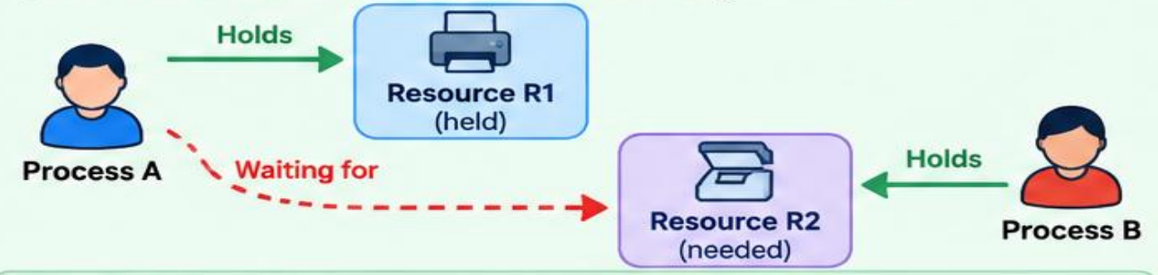
A resource can be used by only one process at a time.



In simple words:
Think of a resource like a single bathroom. Only one person can use it at a time.

2 Hold and Wait

A process holds one resource while waiting for another.



In simple words:
You have a book in your hand, but you are waiting for a pen to write. You are holding one resource and waiting for another.

3 No Preemption

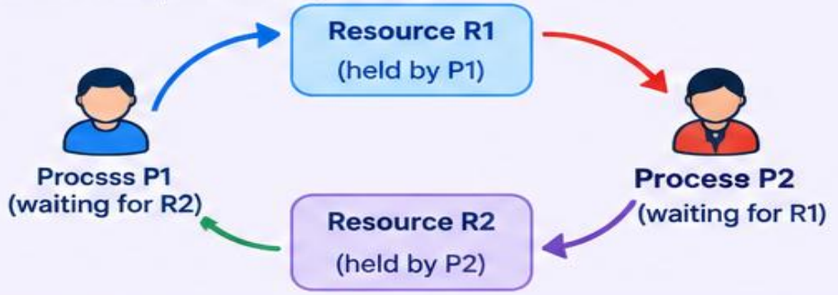
A resource cannot be forcibly taken from a process.



In simple words:
You cannot just snatch a book from someone's hand. The person has to release it willingly.

4 Circular Wait

Processes form a circular chain in which each process waits for a resource held by the next process.



In simple words:
It's like a circle – A waits for B, B waits for C, and C waits for A.

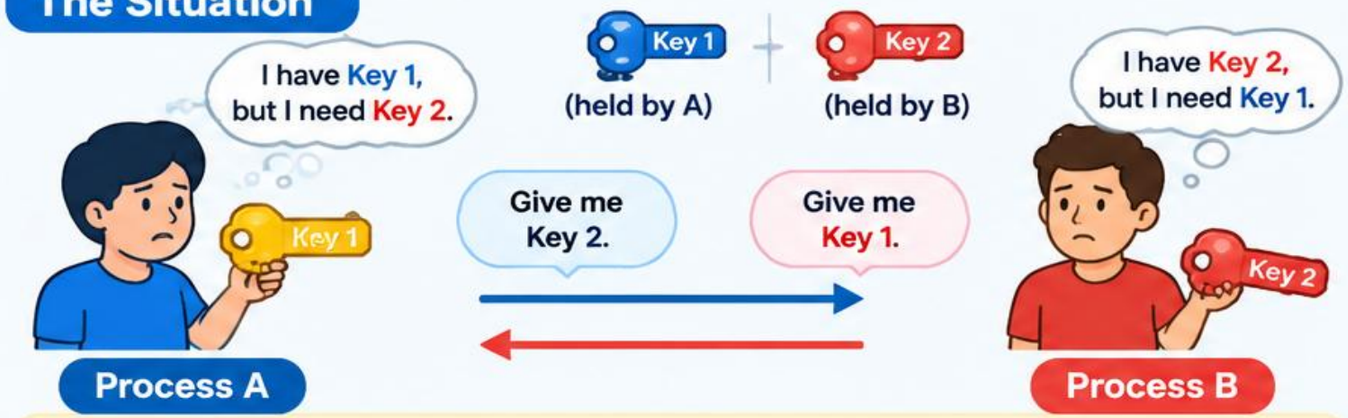
If all 4 conditions happen together → Deadlock!

MIT-WPU

Deadlock – The Two People and Two Keys Example

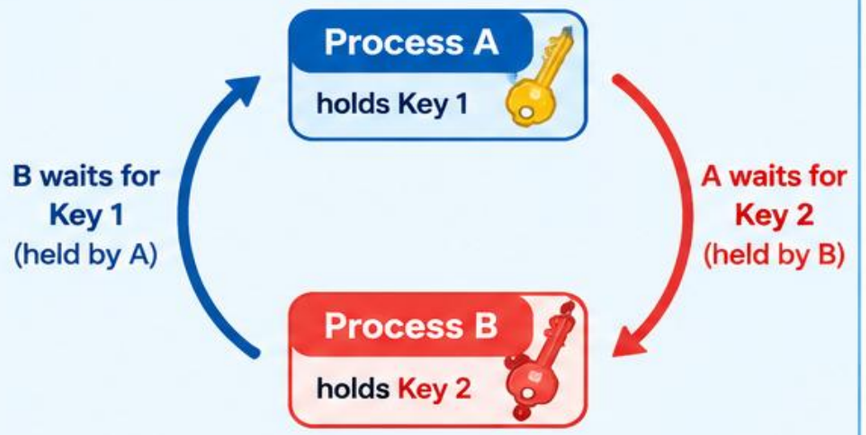
Think of two people holding two different keys.

The Situation



Neither gives up their key, so both keep waiting forever. → **Deadlock!**

The Deadlock (Circular Wait)



The Four Necessary Conditions of Deadlock

1 Mutual Exclusion

One key → One person at a time.



(only A can use it)

A resource (key) can be used by only one process at a time.

2 Hold and Wait

A keeps Key 1 + wants Key 2.



A process holds one resource while waiting for another.

3 No Preemption

A cannot take Key 2 from B.

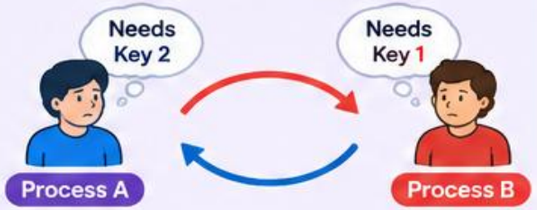


Cannot be forcibly taken

A resource cannot be forcibly taken from a process.

4 Circular Wait

A waits for B, B waits for A.

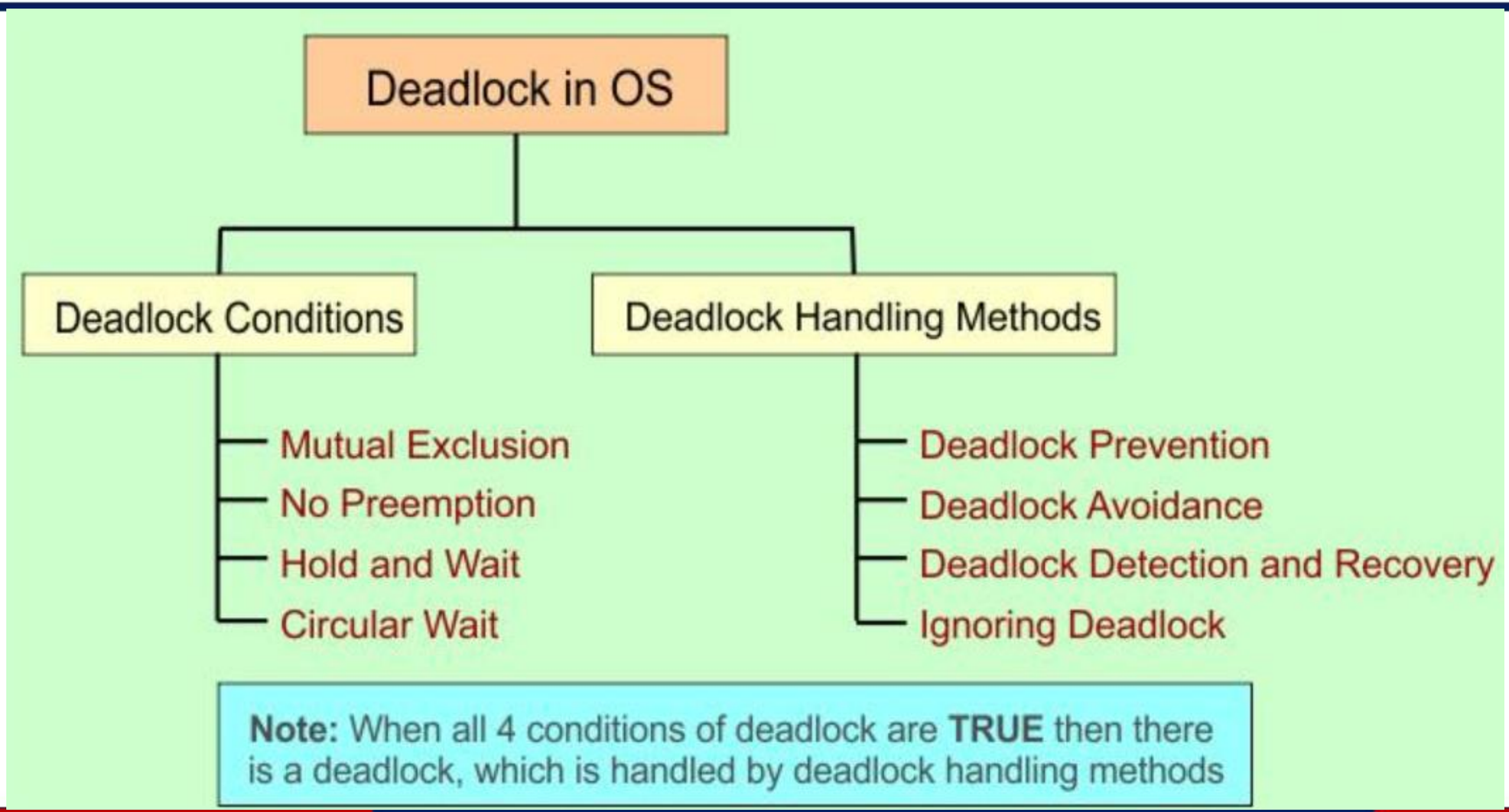


Processes form a circular chain where each waits for a resource held by the next process.

All four conditions together → Deadlock!

#Edu

Methods for Handling Deadlocks



Methods for Handling Deadlocks

- Ensure that the system will ***never*** enter a deadlock state:
  - Deadlock prevention
  - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

There are four approaches to dealing with deadlocks.

```
1. Deadlock Prevention
2. Deadlock avoidance (Banker's Algorithm)
3. Deadlock detection & recovery
4. Deadlock Ignorance (Ostrich Method)
```

Methods for Handling Deadlocks

Invalidate one of four necessary conditions

1. Deadlock prevention ensures that at least one of the four necessary conditions for deadlock can never occurs.

- The four necessary conditions are:
  1. . **Mutual Exclusion** – A resource can be used by only one process at a time.
  2. . **Hold and Wait** – A process holds one resource while waiting for another.
  3. .**No Preemption** – A resource cannot be forcibly taken from a process.
  4. . **Circular Wait** – Processes form a circular chain in which each process waits for a resource held by the next process.

Condition	Prevention Technique
Mutual Exclusion	Make resources shareable whenever possible
Hold and Wait	Require processes to request all resources at once
No Preemption	Allow resources to be forcibly taken when appropriate
Circular Wait	Impose a fixed ordering on resource requests

Deadlock Prevention

Prevent the conditions → Avoid deadlocks → Keep the system running

1 What is Deadlock Prevention?

- **Deadlock prevention** means **preventing deadlocks** by ensuring that at least one of the four necessary conditions for deadlock never occurs.
- It is achieved by restricting resource allocation based on certain rules.

Key idea: Do not allow the system to enter a state where all four conditions can hold simultaneously.

Four Necessary Conditions (for Deadlock)

1. **Mutual Exclusion**
   A resource can be used by only one process at a time.
2. **Hold and Wait**
   A process holds at least one resource while waiting for another.
3. **No Preemption**
   A resource cannot be forcibly taken from a process.
4. **Circular Wait**
   Processes form a circular chain, each waiting for a resource held by the next.

How It Works (Step-by-Step)

1. **Identify the conditions**
   Check which of the four conditions can be prevented.
2. **Apply restrictions**
   Use rules like resource ordering, allocate all resources at once, etc.
3. **Avoid circular wait**
   Impose a fixed order for resource requests.
4. **Result**
   Deadlock cannot occur, but resource utilization may be reduced.

Common Techniques

1. Mutual Exclusion Prevention

Make resources shareable whenever possible (e.g., spooling, shared read-only files).

2. Hold and Wait Prevention

Require processes to request all resources at once (or release current ones first).

3. No Preemption Prevention

Allow resources to be forcibly taken from a process when needed.

4. Circular Wait Prevention

Impose a fixed ordering on resource requests (e.g., $R_1 < R_2 < R_3$ ...). Processes must request in increasing order.

Example: Resource Ordering

If resources are ordered **R1 < R2 < R3**, then a process must request them in increasing order.

Valid request:

P1: $R_1 \rightarrow R_3$ P2: $R_2 \rightarrow R_4$

Invalid request:

P1: $R_3 \rightarrow R_1$ (not allowed)

Key Points to Remember

- Deadlock prevention is proactive (stops it from happening).
- It restricts resource allocation based on the four conditions.
- It may reduce resource utilization and cause unnecessary waiting.
- It is simple but not always practical in dynamic systems.

Drawback

- Resource utilization may be reduced as some resources cannot be shared.
- Processes may have to wait even when resources are available.

In Short

Deadlock prevention makes deadlock impossible by breaking at least one of the four necessary conditions.

Deadlock Prevention

Deadlocks can be prevented by preventing at least one of the four required conditions:

Mutual Exclusion –

- Not required for sharable resources (e.g., read-only files);
- must hold for non-sharable resources
- Unfortunately, some resources, such as printers and tape drives, require exclusive access by a single process.

Hold and Wait –

- To prevent this condition processes Must guarantee that whenever a process requests a resource, it does not hold any other resources.
- There are several possibilities for this:
  - Require that all processes request all resources at one time.
  - Require that processes holding resources must release them before requesting new resources

Deadlock Prevention (Cont.)

No Preemption –

- If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
- Preempted resources are added to the list of resources for which the process is waiting
- Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

Circular Wait – One way to avoid circular wait is to number all resources, and to require that processes request resources only in strictly increasing (or decreasing) order.

- In other words, in order to request resource Rj, a process must first release all Ri such that i >= j.

high level process not wait for low level process

Deadlock Avoidance

1 What is Deadlock Avoidance?

- Deadlock avoidance allows the four conditions to exist but makes resource-allocation decisions carefully so that the system never enters an ***unsafe state***.
- The operating system needs information about the **maximum resource requirements** of processes.
- The most famous deadlock-avoidance algorithm is the **Banker's Algorithm**.

2 Banker's Algorithm

The Banker's Algorithm checks whether granting a resource request will leave the system in a ***safe state***.



3 Safe State

A system is in a ***safe state*** if there is at least one sequence in which all processes can complete successfully.

Example of a safe sequence:

P2 → P1 → P3 → P4

If such a sequence exists, the system can safely allocate resources according to that sequence.

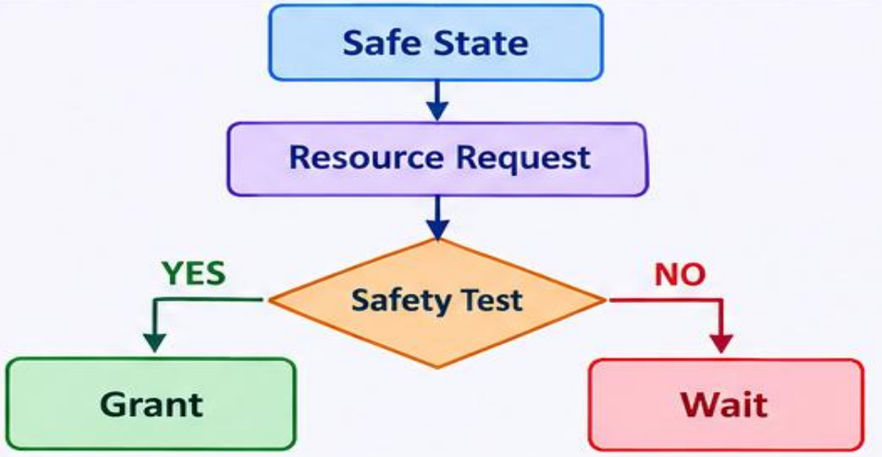
4 Unsafe State

An unsafe state ***does not necessarily*** mean that deadlock has already occurred.

It means that the system has entered a state where deadlock ***could potentially occur***.

! In an unsafe state, the system is not guaranteed to be able to allocate future resources safely.

5 Resource Request Flow



Deadlock Detection

Let it happen → Detect it → Recover it

What is Deadlock Detection?

- **Deadlock detection** allows deadlocks to occur and then detects them using special algorithms.

In addition to the performance hit of constantly checking for deadlocks, a policy is required for removing them from deadlocks, and there is potential for lost work when processes must be aborted or have their resources preempted.

Detection Flow



1 Single Instance of Each Resource

Wait-for Graph

- Used when each resource has only one instance.
- A directed graph is created where an edge from P_i to P_j means P_i is waiting for a resource held by P_j.
- A cycle in the graph indicates a deadlock.

Example (Cycle)



Cycle = Deadlock

2 Multiple Instances of Resources

Detection Algorithm

- Checks the system's state using:

1. **Available** (free instances of each resource type)
2. **Allocation** (resources currently allocated to each process)
3. **Need / Outstanding Requests** (remaining resources each process needs)

Determines whether any processes are deadlocked.

Key Point

- Detection may incur overhead due to continuous checking and graph/algorithm execution.
- Recovery can cause lost work when processes are aborted or resources are preempted.

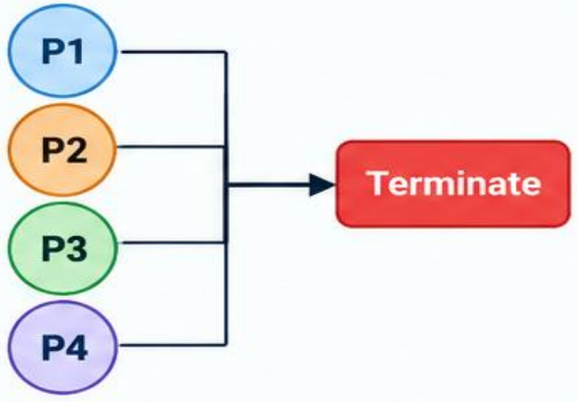
4. Deadlock Recovery

Once a deadlock has been detected, the operating system must recover from it.
There are two common approaches.

A. Process Termination

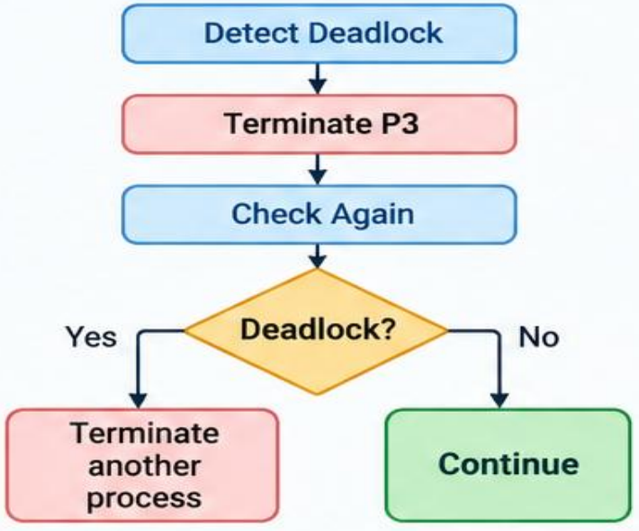
The operating system terminates one or more processes involved in the deadlock.

Option 1: Terminate All Deadlocked Processes



This is simple but can result in significant loss of work.

Option 2: Terminate One Process at a Time

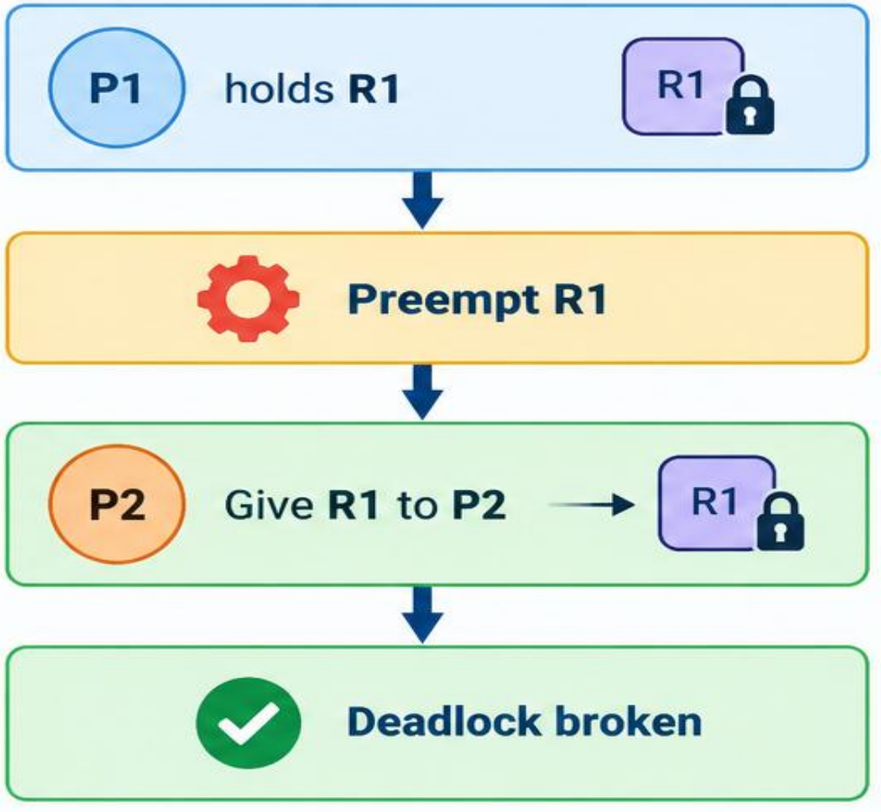


Factors for Selecting a Process for Termination

- Process priority
- Amount of work completed
- Number of resources held
- Resources still required
- Cost of restarting the process

B. Resource Preemption

The operating system temporarily takes a resource from one process and gives it to another process.



Thank You !!!